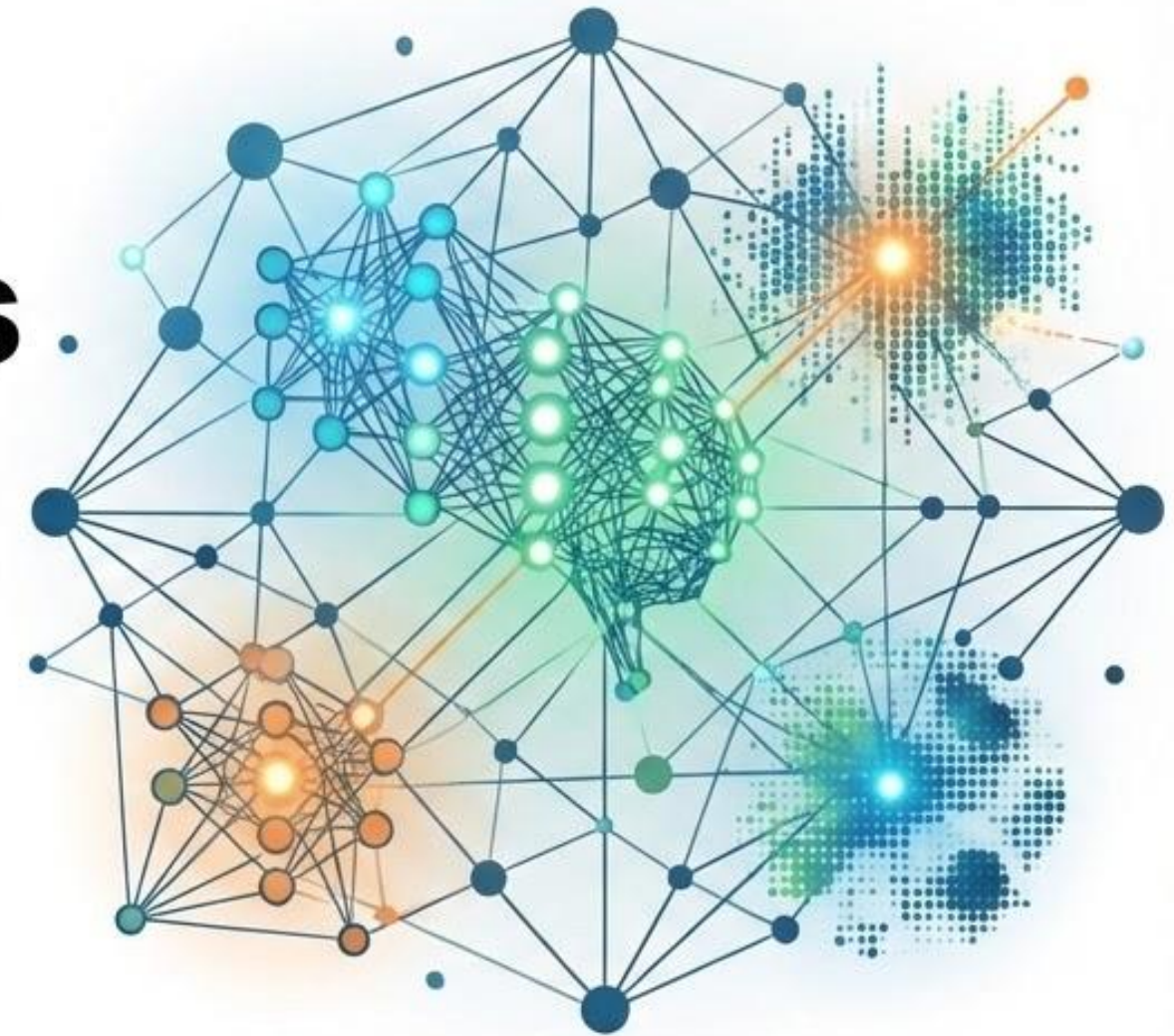


Diplomatura en python para la ciencia de datos y machine learning

Módulo 1 – Desarrollo
Avanzado en Python



4- Manejo de Errores y Calidad de Código

¿Qué es una excepción?

Una excepción es un error que ocurre durante la ejecución del programa.

Si no se maneja



- El programa se detiene.
- Se pierde control del flujo.

```
print(10 / 0)
```

ZeroDivisionError: division by zero

Objetivo

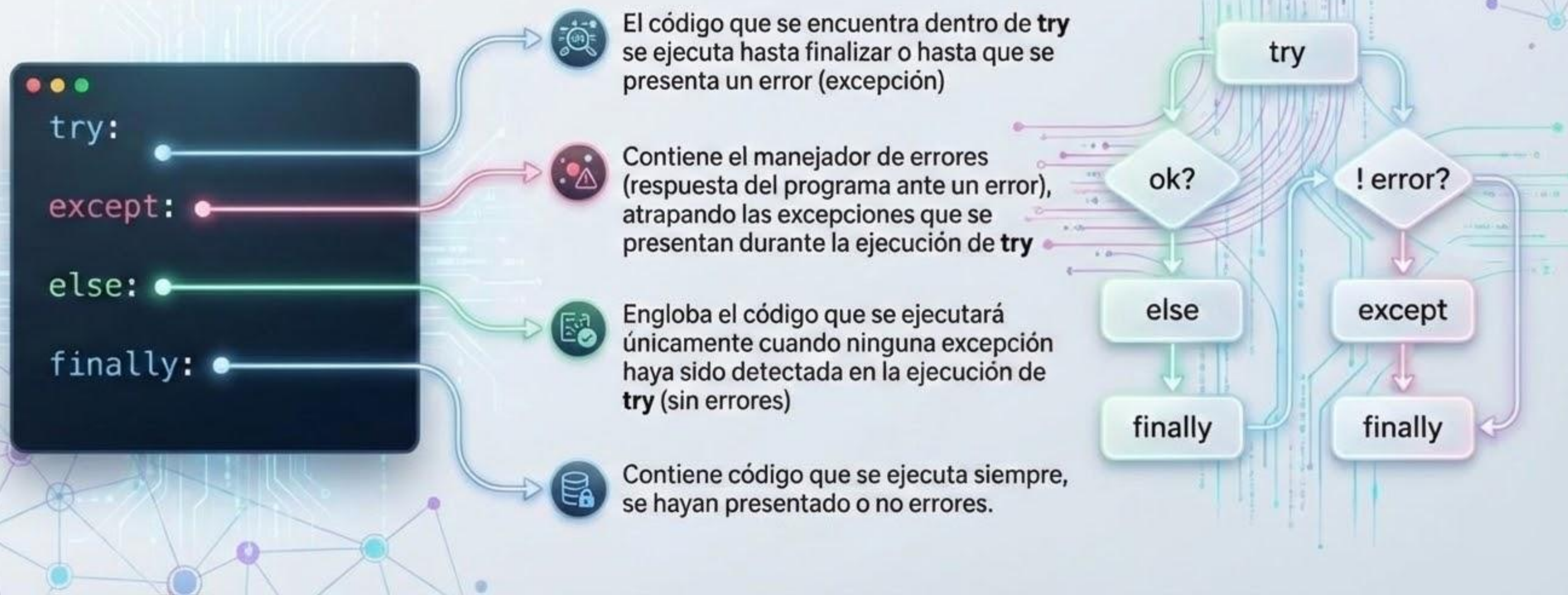


Controlar errores sin que el programa colapse.

Manejo de Excepciones con `try` y `except`

Estructura Básica

El bloque `try-except` es fundamental. Permite capturar y manejar excepciones.

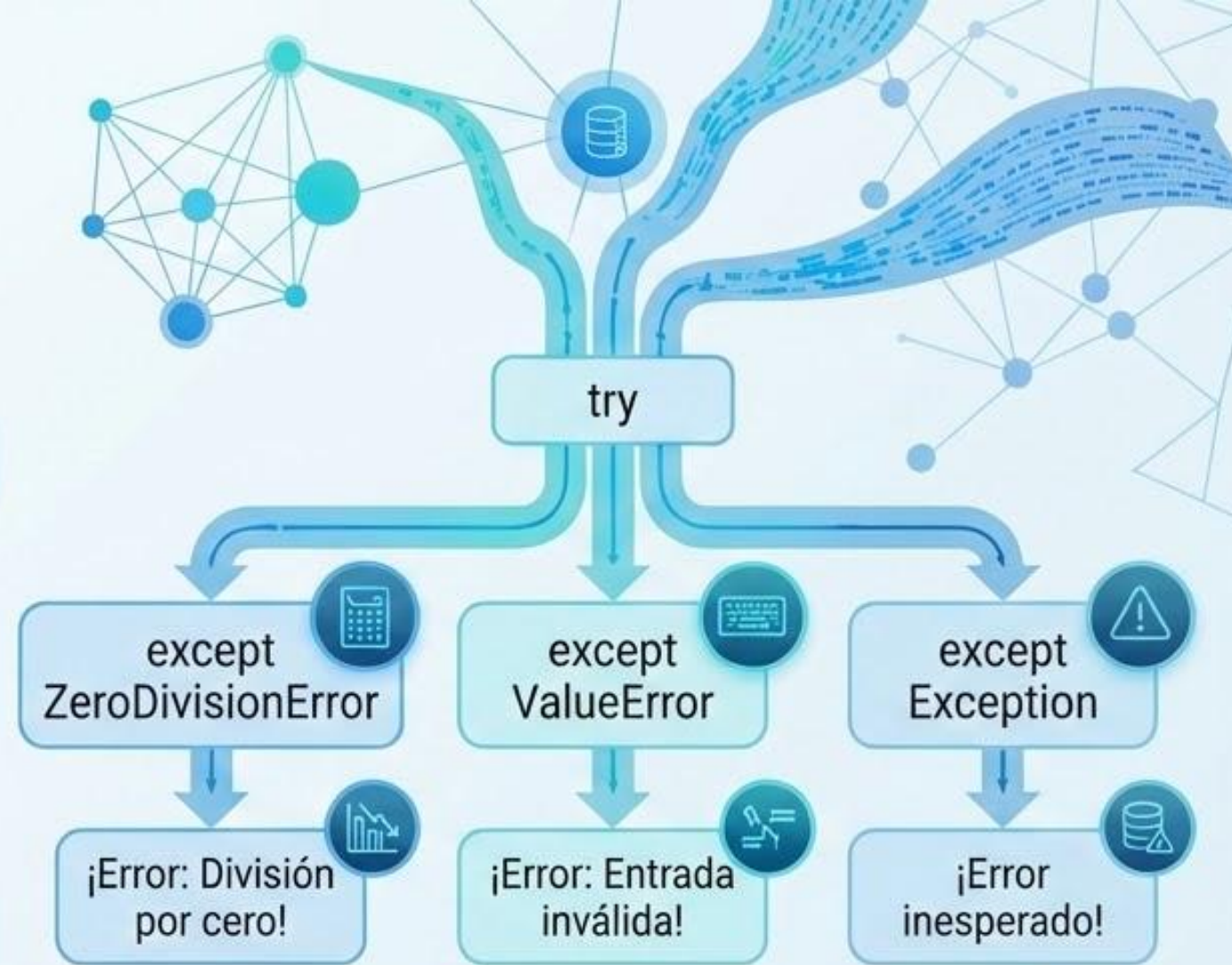


Múltiples bloques `except`

Captura selectiva de errores.

- **try:**
...
except Exception1:
...
except Exception2:
...
except Exception3:
...
- Se pueden usar varios bloques **`except`** para manejar diferentes tipos de excepciones de forma específica. El orden importa: Python buscará el primer bloque **`except`** que coincida con el error lanzado.

```
try:  
    numero = int(input("Ingrese un número: "));  
    resultado = 10 / numero;  
except ZeroDivisionError:  
    print(";Error: No se puede dividir por cero!");  
except ValueError:  
    print(";Error: Entrada inválida. Debe ser un número entero.");  
except Exception as e:  
    print(f";Error inesperado: {e}")
```



Principales Excepciones en Python



TypeError: Ocurre cuando se aplica una operación o función a un dato del tipo inapropiado.



ZeroDivisionError: Ocurre cuando se intenta dividir por cero.



OverflowError: Ocurre cuando un cálculo excede el límite para un tipo de dato numérico.



IndexError: Ocurre cuando se intenta acceder a una secuencia con un índice que no existe.



KeyError: Ocurre cuando se intenta acceder a un diccionario con una clave que no existe.



FileNotFoundError: Ocurre cuando se intenta acceder a un fichero que no existe en la ruta indicada.



ImportError: Ocurre cuando falla la importación de un módulo.

La instrucción `raise`

Permite lanzar una excepción manualmente.



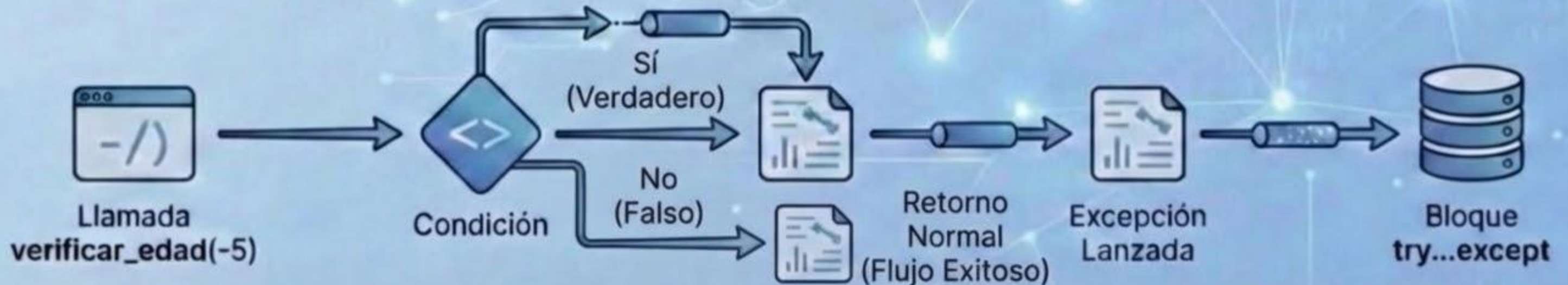
Se utiliza para provocar una excepción intencionalmente con el fin de verificar_edad.



Puede lanzar una excepción manualmente. Útil para señalar un error o condición.

codan.py X

```
1 def verificar_edad(edad):  
2     if edad < 0:  
3         raise ValueError("Age cannot be negative")  
4     return f"Age {edad} is valid"  
5  
6 try:  
7     print(verificar_edad(-5))  
8 except ValueError as e:  
9     print(f"Error: {e}")  
10
```



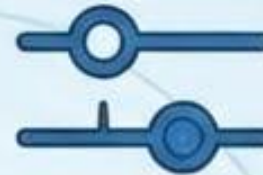
Ejemplos de Validación de Datos



1. Validación de Tipo (Numérico)

```
def pedir_numero():  
    try:  
        valor = int(input('Ingrese un número: '))  
        return valor  
    except ValueError:  
        print('Error: Debe ser un número entero.')
```

Previene TypeErrors y
ValueErrors



2. Validación de Rango

```
def establecer_puntuacion(puntos):  
    if 0 <= puntos <= 100:  
        print('Puntuación válida.')    else:  
        raise ValueError('Puntuación fuera de  
rango 0-100.')
```

Asegura valores lógicos y lanza
excepciones personalizadas

Manejo de errores en archivos



```
try:
    with open('datos.csv') as f:
        contenido = f.read()
except FileNotFoundError:
    print('El archivo no existe')
except PermissionError:
    print('No tiene permisos')
```



Clave en Ciencia de Datos: Los archivos fallan más de lo que creemos.

Logging: Más allá del error en pantalla

❌ El Problema

- ❌ **Inseguridad:** Mostrar errores técnicos al usuario es un riesgo.
- ❌ **Volatilidad:** Los mensajes en consola desaparecen al cerrar el programa.
- ❌ **Invisibilidad:** En servidores (back-end), nadie está mirando la pantalla.

✅ La Solución

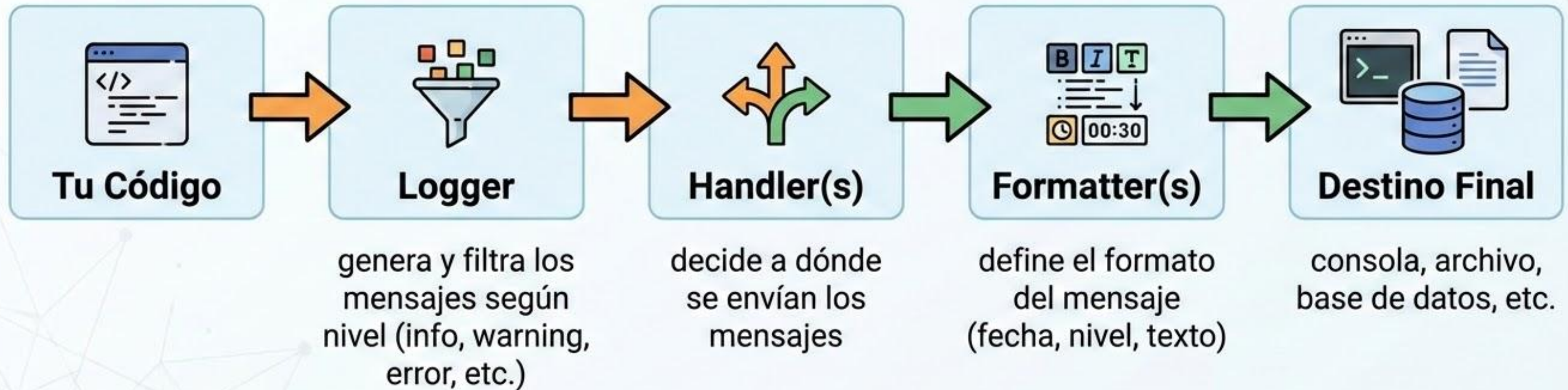
- ✅ **Persistencia:** Redirigir el error a un archivo de registro (.log).
- ✅ **Estructura:** Incluir automáticamente fecha, hora y nivel de gravedad.

🎯 Propósito

- 🎯 **Trazabilidad:** Saber exactamente qué pasó en el pasado.
- 🎯 **Diagnóstico:** Reparar fallos en producción sin replicarlos manualmente.
- 🎯 **Auditoría:** Mantener un historial de salud del sistema.

¿Qué es logging?

Logging es el proceso de registrar eventos que ocurren durante la ejecución de un programa, útil para depuración, monitoreo y auditoría.



Niveles Estándar de Log



INFO

Flujo normal del sistema (ej. "Usuario inició sesión").



WARNING

Algo inesperado ocurrió pero el sistema sigue vivo.



ERROR

Una operación falló (aquí es donde suelen vivir las excepciones).



FATAL/CRITICAL

El sistema o un componente principal se ha caído.



Ejemplo de Logging



```
import logging

# Configuración básica del logger
logging.basicConfig(level=logging.ERROR, filename='app_errors.log')

def dividir_por_usuario(divisor):
    try:
        resultado = 100 / divisor
        return resultado
    except ZeroDivisionError as e:
        # No solo atrapamos el error, lo documentamos con contexto
        logging.error(f"Fallo en división: El usuario ingresó {divisor}. Error: {e}")
        return "No puedes dividir por cero, inténtalo de nuevo."
    except Exception as e:
        # Captura errores inesperados con el stack trace completo
        logging.exception("Ocurrió un error no previsto en el cálculo.")
        raise
```

Tips



Evita el "Log and Throw"

Un error común es registrar el log y luego lanzar la excepción de nuevo para que otro nivel la registre otra vez. **Eso ensucia el archivo de logs** con duplicados. Registra el log solo donde decidas manejar el error.



Datos Sensibles

¡Jamás registres contraseñas, tokens o datos personales (PII) en los logs!